

University of Central Punjab

Faculty of Information Technology

Object Oriented Programming

Fall 2025

Lab 9	
Topic	Classes in C++, access specifiers, objects, methods, constructors, destructors, Inheritance (Single, Multiple, Multi-Level)
Objective	Making students familiarize with the concepts of constructors, destructors, Use inheritance (Base class and Derived Class).
CLO	Apply object-oriented programming principles to implement real-world problems.

Instructions:

- Indent your code.
- Comment your code.
- Use meaningful variable names.
- Plan your code carefully on a piece of paper before you implement it.
- Name of the program should be same as the task name. i.e. the first program should be Task_1.cpp
- **void main() is not allowed. Use int main()**
- **You are not allowed to use any built-in functions**
- **You are required to follow the naming conventions as follow:**
 - o **Variables:** firstName; (no underscores allowed)
 - o **Function:** getName(); (no underscores allowed)
 - o **ClassName:** BankAccount (no underscores allowed)

Students are required to complete the following tasks in lab timings.

Task 1:

Implement Inheritance.

- Base Class: Animal

Create a class named **Animal** with the following attributes and methods:

Attributes:

- **name (char *)** – Represents the name of the animal.
- **age (int)** – Represents the age of the animal.

Functions:

- a) **display()** Function to display a message about the animal class, “Animal object created”.

- Derived Class: Cat

Create a class named **Cat** that inherits the class ‘Animal’.

Functions:

- b) In main, try calling superclass **display()** function to print a message for Animal class object and Cat class object.

Task 2:

Implement Inheritance with Constructors.

- Base Class: Vehicle

Create a class named **Vehicle** with the following attributes and methods:

Attributes:

- **regNo (int)** – Represents the registration number of the vehicle.
- **brand (char *)** – Represents the manufacturer of the vehicle.

Functions:

- a) **Constructor:** A default constructor that prints a message "Vehicle created."

- Derived Class: Car

Create a class named **Car** that inherits from Vehicle with the following:

Functions:

- b) **Constructor:** A default constructor for **Car** class that prints a message "Car created."
- c) In main, create an object of the Car class.
- d) Observe the constructor messages to ensure proper inheritance.

Task 3:

Implement Parameterized Constructor in Superclass (Shapes)

- Base Class: Shape

Create a class named **Shape** with the following attributes and methods:

Attributes:

- **color (char *)** – Represents the color of the shape.
- **sides (int)** – Represents the number of sides of the shape.

Functions:

- a) **Constructor:** Implement a parameterized constructor that takes color and sides as parameters and initializes the attributes.

- Derived Class: Rectangle

Create a class named **Rectangle** that inherits from **Shape** with the following:

Attributes:

- **length (double)** – Represents the length of the rectangle.
- **width (double)** – Represents the width of the rectangle.

Functions:

- b) **Constructor:** Write a constructor in **Rectangle** class, that initializes color, sides, length, and width. Since **Shape** does not have a default constructor, you must call the parameterized constructor of the **Shape** class using an initializer list and provide the values for color and sides.
- c) In main, create an object of the **Rectangle** class, providing values for color, length, and width.
- d) Verify the output to ensure that the constructor initializes correctly.

Task 4:

Implement Constant Member in Superclass (Appliances)

- Base Class: Appliance

Create a class named **Appliance** with the following attributes and methods:

Attributes:

- **const brand (char *)** – Represents the brand name of the appliance (constant).

Functions:

- a) **Constructor:** A parameterized constructor to initialize the brand name.

- Derived Class: AirConditioner

Create a class named **AirConditioner** that inherits from Appliance with the following:

Attributes:

- **Price (double)** – Represents the price of the air conditioner.
- **warranty (int)** – Represents the warranty period in years.

Functions:

- b) In main, create an object of the **AirConditioner** class.
- c) **Constructor:** Implement constructor that initializes brand name, price, and warranty.
- d) Print the values of brand name, price, and warranty to verify the output.

Task 5:

Implement Protected Inheritance with Employee and Manager

- Base Class: Employee

Attributes:

- a) **name (char*)** – Represents the name of the employee.
- b) **emplID (int)** – Represents the employee ID.
- c) **Salary (double)** – Represents the salary of the employee.

Functions:

- a) **Constructor:** Implement a parameterized constructor to initialize the name, emplID and salary.
- b) Implement **Getter and Setter functions, to set and retrieve values of the above mentioned attributes.**

- Derived Class: Manager

Attributes:

- **teamSize (int)** – Represents the number of employees managed by the manager.
- **budget (double)** – Represents the budget for the manager's team.

Functions:

- c) **Constructor:** Implement a constructor to initialize the teamSize, budget, and call the base class constructor as well.
- d) Implement **Getter and Setter functions for Manager Class attributes.**
- e) **display():** Implement a display function that prints values of attributes from both classes.
- f) In main, create an object of the **Manager class**, providing values for Employee and Manager attributes.
- g) Try calling the getter functions to retrieve and display all the attributes.
- h) Call the **display()** function also to demonstrate the functionality.

Task 6:

Implement Private Inheritance with Employee and Manager

Use the above task in 'Task 5' to implement private inheritance and check its various functionalities.

- Base Class: Employee

Attributes:

- d) **name (char*)** – Represents the name of the employee.
- e) **emplID (int)** – Represents the employee ID.
- f) **Salary (double)** – Represents the salary of the employee.

Functions:

- a) **Constructor:** Implement a parameterized constructor to initialize the name, emplID and salary.
- b) Implement **Getter and Setter functions, to set and retrieve values of the above mentioned attributes.**

- Derived Class: Manager

Attributes:

- **teamSize (int)** – Represents the number of employees managed by the manager.
- **budget (double)** – Represents the budget for the manager's team.

Functions:

- c) **Constructor:** Implement a constructor to initialize the teamSize, budget, and call the base class constructor as well.
- d) Implement **Getter and Setter functions for Manager Class attributes.**
- e) **display():** Implement a display function that prints values of attributes from both classes.

Observe Access Limitations:

- f) In main function, after creating object of the **Manager** class, try to access the **name, emplID, and salary** members directly, and observe the behavior.
- g) **Next, use the getter functions** to retrieve these values.

- h) Consider how the behavior would change if you used public or protected inheritance instead. Modify the inheritance in the Manager class and repeat the access tests, to observe the differences.
- i) Call the **display()** function also to demonstrate the functionality.

Task 7:

Multi-Level Inheritance (ElectronicDevice -> SmartDevice -> SmartPhone)

- **Base Class: ElectronicDevice**

Attributes:

- **brand (char *)** - Represents the brand of the electronic device.
- **warrantyPeriod (int)** - Warranty period in years.

Functions:

- a) **Constructor:** Implement a parameterized constructor to initialize brand and warrantyPeriod.
- b) Implement **Getter and Setter functions** for brand and warrantyPeriod.
- c) **display():** Displays the brand and warranty period of the electronic device.

- **Intermediate Class: SmartDevice**

Attributes:

- **supportBattery (bool)** - If the device supports battery or not (e.g, in smart TV or laptop)

Functions:

- d) **Constructor:** Implement a parameterized constructor to initialize supportBattery and calls the ElectronicDevice class constructor.

e) Implement **Getter and Setter functions** for supportBattery.

- **Derived Class: SmartPhone**

Attributes:

- **storageCapacity (int)** – Storage capacity in GB.
- **cameraResolution (float)** – Camera resolution in megapixels.
- **price (double)** – Price of the device.

Functions:

- f) **Constructor:** Implement a parameterized constructor to initialize storageCapacity, cameraResolution and price, and calls the constructors of both ElectronicDevice and SmartDevice classes.
- g) Implement **Getter and Setter functions** for storageCapacity, cameraResolution and price.

- **Main Function Tasks:**

- h) In the main function, create an object of the SmartPhone class, providing values for all attributes.
- i) Call the display function of the base class using objects of all classes.

Task 8:

Implement Multi-level Inheritance (Shape -> 2DShape -> Rectangle)

- **Base Class: Shape**

Attributes:

- **color (char *)** – Represents the color of the shape.

Functions:

- a) **Constructor:** Implement a parameterized constructor to initialize the attribute color.
- b) Implement **Getter and Setter functions** to initialize and retrieve the **color** attribute of Shape class.
- c) **display():** Implement a display function that prints the color of the shape.

• *Derived Class: 2DShape*

Attributes:

- **sides (int)** – Represents the total number of sides of the shape.

Functions:

- d) **Constructor:** Implement a parameterized constructor to initialize color and sides. Call the Shape class constructor.
- e) Implement **Getter and Setter functions** to initialize and retrieve the value of sides.

• *Derived Class: Rectangle*

Attributes:

- **length (double)** – Represents the length of the rectangle.
- **width (double)** – Represents the width of the rectangle.

Functions:

- f) **Constructor:** Implement a parameterized constructor to initialize color, sides, length, and width. Call the constructors of both Shape and 2DShape.
- g) Implement **Getter and Setter functions** to initialize and retrieve the attributes of Car class.
- h) **calculate_area():** Implement a method to calculate and return the area of the rectangle.

- i) **calculate_perimeter()**: Implement a method to calculate and return the perimeter of the rectangle.

- **In the main function**

- a) Create objects of the Rectangle class and provide the values for their required attributes.
- b) Try calling the **display()** function of the base class using objects of all three classes.

Task 9:

Vehicle + RoadVehicle -> Car

- **Base Class: Vehicle**

Attributes:

- **regNo (int)** – Represents registration number of vehicle
- **brand (char *)** – Represents the brand of the vehicle.

Functions:

- a) **Constructor**: Parameterized constructor to initialize vehicle regNo and brand.
- b) Implement **Getter and Setter** functions to initialize and retrieve the attributes of Vehicle class.
- c) **display()**: Implement a display function that prints the regNo and brand name of the vehicle.

- **Base Class: RoadVehicle**

Attributes:

- **numberOfWheels (int)** - Represents the number of wheels.

Functions:

- d) **Constructor:** Implement a parameterized constructor to initialize **numberOfWheels** and **fuelType** for the **RoadVehicle** class.
- e) Implement **Getter and Setter** functions to initialize and retrieve the attributes of **RoadVehicle** class.

• **Derived Class: Car**

Attributes:

- **color (char *)** - Represents the car color.
- **miles (int)** - Represents the miles travelled by the car.

Functions:

- f) **Constructor:** Write a constructor in the **Car** class that initializes **regNo**, **brand**, **numberOfWheels**, **color** and **miles**. Call the constructors of both **Vehicle** and **RoadVehicle** class.
- g) Implement **Getter and Setter functions** to initialize and retrieve the attributes of **Car** class.
- h) In **main**, create an object of the **Car** class, providing values for **Car**, **RoadVehicle** and **Vehicle** attributes.
- i) Try calling the getter functions for all classes using **Car** object, to retrieve and display the values of all attributes.

Task 10:

Multi-Level Inheritance (Teacher + Researcher → Professor)

Base Class: Teacher

Attributes:

- **name (char *)** – Name of the teacher.
- **subject (char *)** – Subject taught by the teacher.

Functions:

- a) **Constructor** to initialize name, subject.
- b) **displayTeacher()** – Prints the teacher's name, subject.

Base Class: Researcher

Attributes:

- **researchArea (char *)** – Area of research (e.g., AI, Quantum Physics).
- **publications (int)** – Number of research papers or publications.

Functions:

- c) **Constructor:** to initialize name, subject.
- d) **displayResearcher()** – Displays the research area and publications.

Derived Class: Professor (inherits from Teacher & Researcher)

Attributes:

- **department (char *)** – Department the professor belongs to (e.g., Computer Science, Physics).
- **teachingLoad (int)** – Number of courses taught per semester.

Functions:

- e) **Constructor:** to initialize department, and teachingLoad. Calls the constructors of both Teacher and Researcher.
- f) **displayProfessor()** – Displays the department and teaching load, in addition to calling the displayResearcher() and displayTeacher() function of the Researcher class.